Block Away 3D

Version 1.0.0 · SatisVizion · Unity 6000.0.62f1 · URP 17.0.4

Drag to rotate the block and tap an arrow whose path to the edge is clear. Blocked arrows bump back and cost a heart. Clear every arrow to finish the level.

First run

1. Install Unity **6000.0.62f1** in Unity Hub. For Android add matching Android Build Support, SDK, NDK and OpenJDK; for browser builds add WebGL Build Support.
2. Open the supplied standalone project, or create a Unity 6 **URP** project before importing the buyer package. Required packages: URP **17.0.4**, uGUI **2.0.0**, Input System **1.14.2**, the standard Unity modules, **ScreenCapture** and **AndroidJNI**. The standalone project includes its manifest and lock file.
3. Import the complete product folder. Keep its required subfolders together: **Assets**/**SatisVizionBlockAway**/**Games**/**BlockAway**, **Assets**/**SatisVizionBlockAway**/**HyperCasualLiveOpsKit** and **Assets**/**SatisVizionBlockAway**/**TextMesh Pro**. Keep every `.meta` file and the game's private **Kit** folder; it is required runtime code.
4. Set Edit → Project Settings → Player → Other Settings → **Active Input Handling = Both** and restart Unity if prompted. Run **Tools** → **Block Away 3D** → **Setup** after package import to select the supplied URP settings.
5. Open `Assets/SatisVizionBlockAway/Games/BlockAway/Scenes/BlockAway_Mobile.unity` and press Play. This is the portrait scene. A landscape WebGL scene is also included; use the scene appropriate for your build. Much of the game UI and world are created at runtime.

Gameplay and content

Drag to rotate the block and tap an arrow whose path to the edge is clear. Blocked arrows bump back and cost a heart. Clear every arrow to finish the level.

Rotatable toy blocks, directional arrow paths, frozen arrows, heart limits, solver-backed generation and guided booster introductions.

The source includes **25 authored levels** and seeded procedural continuation. Progress, settings and charges are stored locally with PlayerPrefs. Browser storage depends on the browser profile. The template includes source and editable level data; generated continuation is not advertised as hundreds of individually authored levels.

Boosters

- **Hint** unlocks at level **3**.
- **Pluck** unlocks at level **6**.
- **Scan** unlocks at level **9**.

- **Sweep** unlocks at level **15**.

First-use guidance introduces each booster. Availability depends on the level, current state and remaining charges. Rewarded placements are simulated by default; only a completed reward callback should grant a refill. Use the DEV panel to unlock boosters for authoring, not as a substitute for real progression testing.

Booster usage and charges

The table lists the campaign unlock level and initial grant for each booster. Grants occur through the unlock/progression flow; existing local saves retain their stored inventory.

Booster	Unlock	Starting charges	Effect and input
Hint	3	3	Tap Hint to turn toward a currently removable arrow, then tap that glowing arrow normally. No valid free arrow means no charge is spent.
Pluck	6	2	Tap Pluck, then select an arrow to lift it away, including frozen arrows. This removal costs no move. Tap the booster again to cancel targeting.
Scan	9	2	Tap Scan to highlight arrows that can currently leave in blue. It does not remove them; tap arrows normally.
Sweep	15	1	Tap Sweep to remove all currently free arrows together for one move. It requires at least one eligible arrow.

A completed activation consumes one charge. Targeted boosters spend it when the target action completes. Invalid actions are rejected by the game. Boosters depend on an active playable level; tutorials, popups and current effects can restrict activation. At zero charges, the refill offer awards exactly one charge only after a successful rewarded callback. Cancelling or failing that callback grants nothing. Shipping ads are simulated; production refills require your own separately configured provider. Charges and tutorial flags are saved locally through HzSave/PlayerPrefs. Tune BoostUnlock, BoostStart and the booster handlers in BlockAwayGame.cs; keep UI guidance and actual rule logic consistent.

Manual editor and generation

Open **Tools → Block Away 3D → Level Editor**. Duplicate a level before changing the original tutorial. The authoring file is `Assets/SatisVizionBlockAway/Games/BlockAway/Resources/blockaway/Levels/levels.json`.

The editor exposes Name, Hearts, Difficulty tag and arrow-cell paths on the block faces. It includes frozen-arrow settings, Validate/Validate All and seeded Auto Generate controls. Duplicate a non-tutorial board before editing. Change its Name and, for a simple first edit, increase Hearts within the allowed range. Validate, Save and test it. Arrow directions, occupied cells and frozen-arrow relationships must still allow a complete removal order; validate again after any geometry change.

The included editor supports saving, duplication and ordering, and provides generation and validation controls appropriate for this game. Read the actual validation result; a difficulty target is not a guarantee of player success. Use a repeatable seed when comparing generated variants. Test changed boards in Play mode.

First authored level schema

```
{
  "levels": [
    {
      "name": "Level 1",
      "tut": 1,
      "breather": false,
      "hard": 0,
      "difficulty": 1.08,
      "sx": 3,
      "sy": 3,
      "sz": 3,
      "moves": 8,
      "hearts": 3,
      "hint": "Tap an arrow to slide it off the block. Clear them all!",
      "rails": [],
      "gems": [],
      "tiles": [],
      "arrows": [
        {
          "d": 3,
          "ice": 0,
          "c": [
            42,
            43,
            7
          ]
        },
        {
          "d": 0,
          "ice": 0,
          "c": [
            41,
            38,
            37
          ]
        },
        {
          "d": 4,
          "ice": 0,
          "c": [
            21,
            50,
            47
          ]
        },
        {
          "d": 2,
          "ice": 0,
          "c": [
            14,
            13
          ]
        }
      ]
    }
  ]
}
```


Customisation

Game logic is under `Scripts`; presentation and UI are in the game's Game partial classes. Artwork, sound effects, music and levels are under `Resources/blockaway`. Shared UI, audio, ad and haptic helpers live in `Kit/Runtime`; font files and placements are under `Kit/Resources/blockaway/kit`. Keep resource paths unless you update every matching code reference.

Change Company Name, Product Name, Version and the Android application identifier in Player Settings to values you control. Do not publish under `com.satisvizion.blockaway.demo`. Assign your own launcher/adaptive icon layers and review the splash screen, in-game title and other branding. Increase Android Bundle Version Code for each released update. Test UI fit, text and touch targets on an actual phone.

LiveOps, ads and haptics

The publisher-owned **HyperCasual LiveOps Kit** is included. Read its complete kit manual (HTML is also included offline).

Settings: `Assets/SatisVizionBlockAway/HyperCasualLiveOpsKit/Resources/HLK/HLKSettings.asset`. Shipping defaults are **Ads Mode Dummy (0)** and **IAP Mode Disabled (3)**. Banner, interstitial and rewarded placements are simulations. **No Google Mobile Ads, billing or other vendor SDK is included.** Automatic vendor-SDK installers are disabled. The game's own Kit is required code, not an advertising SDK.

To enable a real provider, separately install its official SDK, follow the kit manual, configure your own identifiers, consent and privacy policy, then test the selected platform and all cancellation/failure cases. Do not enable SDK symbols without the corresponding dependency. Optional kit modules are capabilities, not production services already configured for this game. Native vibration needs a supported device; browser haptics and other native services have platform limits.

Development controls

Open DEV, press F2/backquote, or use the kit's top-left multi-tap shortcut. The global panel supports level jumps, booster access, ads and haptics controls. Add `HZ_RELEASE` to scripting defines when you want these development controls excluded from a consumer build. This does not configure real ads or certify store readiness.

Android release

Use the Mobile scene, portrait orientation and ARM64/IL2CPP with matching Unity tools. The provided APK is a **debug-signed, non-development showcase demo**, not a production store release. For release keep Development Build and debugging off, choose your own application identifier and review current store API requirements.

In Player → Publishing Settings create or select **your own keystore and alias**. Securely retain the keystore, alias and passwords for future updates; never redistribute signing secrets. Enable Build App Bundle for a Google Play AAB. Install a release candidate on a physical Android device and check touch, audio, haptics, pause/resume, saves and any enabled provider flows. Complete destination-store ads/data declarations and your privacy policy for the services actually enabled.

WebGL

Build the Mobile scene for a portrait-only demo with the built-in Default template and Brotli decompression fallback. Use **Tools → Block Away 3D → Create Responsive WebGL Page** after building and select the output folder. This generates an original responsive page around the complete Build folder. Serve every loader, framework, data and WASM file over HTTP(S); `file://` is unsupported. Audio normally starts after a first click/tap.

The two-scene landscape-first route supports `?mobile=1` to switch to the following Mobile scene. The included WebGL build menu uses HLK's non-development **local showcase** scope. It supports simulated demos; configure your production providers and resolve the kit build-doctor checks for a real service

release.

Source map and a first edit

All required assets are grouped under `Assets/SatisVizionBlockAway`. Keep the game, its own Kit, HyperCasualLiveOpsKit and TextMesh Pro together.

- `Games/BlockAway/Scenes`: supplied mobile and landscape entry scenes.
- `Games/BlockAway/Scripts`: game-specific rules and presentation.
- `Games/BlockAway/Editor`: game-specific authoring tools; shared build/editor helpers also live in the game's Kit.
- `Games/BlockAway/Resources/blockaway`: editable campaign data, artwork and game-specific audio resources.
- `Games/BlockAway/Kit/Runtime`: required game-framework helpers, including saving, audio, ad/haptic integration and developer controls.
- `HyperCasualLiveOpsKit`: the included publisher-owned service kit and its own manual.
- `TextMesh Pro`: included text resources and retained licence notices.

Main source roles:

- BlockAwayGame and its UI/View partials: input, tutorial, progression and presentation.
- BlockData / BlockJson / BlockLevels: level schema, parsing and campaign loading.
- BlockEngine / BlockGrid: board state and arrow movement rules.
- BlockGen / BlockPlan / BlockSolver: generation, planning and authoring validation.

First edit, step by step: open the game's Level Editor from Tools, select a non-tutorial level, click Duplicate (or Dup), change its Name, run Validate, read the result and click Save. Click Play this level in that same editor to start the selected board. Check the changed name, play the goal and retry once. Exit Play mode before further authoring. For a rule or geometry edit, repeat validation and play the full level; a bot success alone does not certify the human experience. Keep a backup of levels.json before replacing a generated range.

Troubleshooting

- Missing types: install the required packages and import all three roots, preserving `.meta` files.
- Pointer input exception: select Active Input Handling Both and restart Unity.
- Pink materials: use URP 17.0.4 and run the game's Setup command.
- Sparse scene in edit mode: world/UI objects are built at runtime; press Play.
- No real ad revenue: Dummy mode is simulated and no provider SDK is bundled.
- IAP is unavailable: it is deliberately Disabled by default; configure an official billing integration separately if you need it.
- Progress persists: use the development reset controls or deliberately clear this product's PlayerPrefs.

- WebGL does not load from a local file: serve the complete output over HTTP(S).

Licences and support

Retain the font, icon, TextMesh Pro and HLK licences and Third-Party Notices. See the game's `Third-Party Notices.txt` and `AI-DISCLOSURE.txt`. Marketing artwork is distinct from authentic gameplay captures. For product support use the marketplace item support channel or contact SatisVizion at satisvizion@gmail.com.